

Multilevel Feedback Queue (MLFQ)

MLFQ gives new processes the highest priority. Processes that use their CPU allowance move to a lower priority. A periodic boost returns active processes to the top queue.

1. Queues and CPU budgets

The visualizer has one CPU and three ready queues: Q0 is highest priority, followed by Q1, then Q2. Time advances in whole ticks. Processes need only CPU service, with no I/O blocking or cost for switching processes.

Three quantities determine how long a process runs:

<i>Quantity</i>	<i>Meaning</i>
Remaining service	CPU ticks still needed to finish the process.
Quantum (time slice)	CPU ticks allowed in one turn at the current priority.
Allotment	Total CPU ticks allowed at the current priority before demotion.

Quantum and allotment are separate settings. Their defaults happen to be equal:

<i>Queue</i>	<i>Quantum</i>	<i>Allotment</i>
Q0	2	2
Q1	4	4
Q2	8	8

The default boost interval is 10 ticks. Every budget and the boost interval must be a positive whole number. The quantum need not divide the allotment or be smaller than it.

Each process has two usage counters. For example, **Q: 1/2** and **A: 3/4** mean that it has used one tick of its current two-tick turn and three ticks of its four-tick allowance at this priority. Its remaining service is a separate value.

2. Choosing a process

Rule 1. New arrivals join Q0

Append a new process to the tail of Q0 with both usage counters at zero. Simultaneous arrivals enter in input order. The leftmost process in a queue is its head.

Rule 2. Choose the highest-priority ready queue

When the CPU is free, run the head of the highest-priority nonempty queue. Processes within a queue take turns in Round Robin order.

If Q0 is empty, Q1 contains [B, C], and Q2 contains [D], choose B. D waits until both higher-priority queues are empty.

Rule 3. Higher-priority work interrupts lower-priority work

If a higher-priority process becomes ready, preempt the lower-priority running process. Return the interrupted process to the **front** of its own queue and preserve both usage counters. Same-priority arrivals do not interrupt a turn.

For example, A is in Q1 with quantum usage 1/4 and allotment usage 5/6 when B arrives in Q0. A returns to the front of Q1. When A resumes, it still has three quantum ticks and one allotment tick available, unless a boost has reset its budgets in the meantime.

3. Accounting for CPU time**Rule 4. Charge only the process that runs**

For each tick of execution, reduce remaining service by one and increase both quantum usage and allotment usage by one. Waiting uses neither budget.

Rule 5. Completion takes precedence

At the next tick boundary, remove a process whose remaining service has reached zero. It is not rotated, demoted, or boosted, even if any of those events would also occur at that boundary.

A process with one service tick left finishes after one tick, regardless of how much quantum or allotment remains.

Rule 6. Exhausting the allotment ends the priority level

If an unfinished process has used its allotment, append it to the next lower queue and reset both usage counters. Thus Q0 moves to Q1, and Q1 moves to Q2. A process already in Q2 rejoins Q2's tail with fresh budgets.

Allotment exhaustion takes precedence over quantum expiry and can happen partway through a turn. With quantum 2 and allotment 3, a process gets one full two-tick turn and only one tick of its next turn before demotion.

Rule 7. Quantum expiry alone ends only the turn

If the quantum expires but allotment remains, append the unfinished process to the tail of the same queue. Reset quantum usage and preserve allotment usage.

With quantum 1 and allotment 4, a process can take four one-tick turns at that priority before demotion, provided it does not finish or receive a boost first. Returning to the queue does not erase the CPU time already charged to its allotment.

Rule 8. Keep the CPU busy when work is ready

After a turn ends, dispatch normally. If the process is alone, it may immediately run again, either at its existing priority or after demotion. No idle tick is inserted. Idle only when nothing is ready; stop when all processes finish.

4. Priority boosts

Rule 9. Boost waiting and running processes

A boost occurs at positive multiples of the boost interval, never at time 0. For interval 10, these are times 10, 20, 30, and so on.

At a boost:

1. Collect waiting processes from Q0, then Q1, then Q2. Preserve the order within each queue.
2. Stop any process whose turn is still in progress and append it after all the waiting processes.
3. Put the collected processes in Q0 and reset both usage counters. Preserve remaining service.
4. Admit any arrivals at this time, then dispatch from Q0.

This also applies to a process already running in Q0. If nobody else is waiting or arriving, the interrupted process is immediately selected again with fresh budgets.

Example. A boost during an ongoing turn

A is running in Q1 without having exhausted either budget. Q0 is empty, B and C wait in Q1, and D waits in Q2.

<i>Stage</i>	<i>CPU</i>	<i>Q0</i>	<i>Q1</i>	<i>Q2</i>
Before boost	A	Empty	B, C	D
After boost, before dispatch	Free	B, C, D, A	Empty	Empty
After dispatch	B	C, D, A	Empty	Empty

All four processes have zero quantum and allotment usage immediately after the boost. A waits behind the other three; its unfinished service has not changed. If E also arrives at this boundary, it joins after A, giving [B, C, D, A, E] before dispatch.

5. Events at the same boundary

The order below is part of the implemented policy. It matters when expiry, boost, and arrival happen together. In both cases, first account for execution during the preceding tick.

Without a boost

1. Remove the running process if it has finished.
2. Check for allotment exhaustion, then quantum expiry, in any unfinished running process.
3. Add all arrivals at the current time.
4. Requeue the process whose budget expired, applying Rule 6 or Rule 7.
5. If a process is still running, check for higher-priority preemption.
6. Dispatch if the CPU is free.

Arrivals therefore enter before expired work is requeued. An existing waiting process keeps its place within its queue.

With a boost

1. Remove the running process if it has finished.
2. Check for allotment exhaustion, then quantum expiry, and requeue the process if necessary.
3. Perform the boost, appending only a process whose turn is still in progress after the waiting processes.
4. Add all arrivals at the current time.
5. Dispatch from Q0 if any process is ready.

A process whose budget expired has already entered a queue when the boost begins. It takes that queue's position in the collection order; it is not automatically placed last.

Example. Allotment expiry coincides with a boost

A exhausts its Q0 allotment but still needs service. B waits in Q0, C in Q1, and D in Q2.

Stage	Q0	Q1	Q2
After demoting A	B	C, A	D
After boosting	B, C, A, D	Empty	Empty

A is placed behind C in Q1 before the boost. Collecting the queues gives [B, C, A, D], so B runs next. A new arrival E would be appended after D. If A had finished instead, it would leave first and the boost order would be [B, C, D].

6. Worked example: separate quantum and allotment

A and B both arrive at time 0, in that order, and each needs ten service ticks. Set Q0's quantum to 1 and allotment to 4. Leave Q1 and Q2 at their defaults and set the boost interval to 100, so there is no boost during this example.

The first eight ticks are:

A	B	A	B	A	B	A	B	
0	1	2	3	4	5	6	7	8

Figure 1. CPU schedule (time in ticks).

After A's first, second, and third turns, its quantum usage resets to zero while its allotment usage is 1, 2, and 3. At time 7, A has used four CPU ticks in Q0. It moves to Q1 with both counters reset and six service ticks left.

B is still in Q0, so B gets the CPU for the tick from 7 to 8. At time 8, B also moves to Q1 with six service ticks left. Q1 now contains [A, B] before dispatch, so A runs next.